



ADDIS ABABA SCIENCE AND TECHNOLOGY UNIVERSITY

COLLEGE OF ENGINEERING

DEPARTMENT OF SOFTWARE ENGINEERING

INTEGRATED PARKING MANAGEMENT SYSTEM (PMS)

Course: Database Management Systems

Instructor: Yaynshet Medhin

Date: May 22, 2026

Status: Final Submission

Group Number: 2

Repository link: <https://github.com/A-proG-spec/AASTU-DB-Project-Group-2>

GROUP MEMBERS

No.	Name	ID	Role
1	Ahmedin Muhamed	ETS 0116/17	Project Coordinator
2	Anteneh Wondwosen	ETS0186/17	Backend/Implementation Lead
3	Arjuma Tilahun	ETS0190/17	Business Analyst
4	Arsema Negash	ETS0197/17	Database Designer
5	Betibeb Aregahegn	ETS 0300/17	Documentation Lead
6	Arsema Desalegn	ETS0196/17	Testing & Presentation Lead

TABLE OF CONTENTS

1. Executive Summary	1
2. Introduction & Problem Statement	2
3. Project Objectives	2
4. System Requirements Analysis	3
5. Data Collection Methodology	4
6. Entity-Relationship (ER) Diagram Logic	4
7. Relational Database Schema Design	5
8. Database Normalization & Integrity	7
9. Data Dictionary: User & Vehicle	8
10.Data Dictionary: ParkingLot & ParkingSlot	9
11.Data Dictionary: Booking & Reservation	10
12.Data Dictionary: Payment & Penalty	9
13.Data Dictionary: Staff & SalaryPayment	10
14.Implementation: DDL and DML	11
15.Analysis: Information Retrieval	11
16.Analysis: Financial Reporting	13
17.Analysis: Operational Metrics	14
18.Analysis: Behavioral Patterns	15
19.VIP Program Modeling	16
20.Security & Access Control	17
21.Scalability & Future Work	18
22.Conclusion & Recommendations	19
23.References	21
24.Appendix: Complete SQL Scripts	22

1. Excutive summary

This project is about creating a database for a Parking Management System (PMS).

The system helps manage ten things:

- User
- Booking
- Payment
- Penalty
- Vehicle
- Reservation
- Parking Lot
- Salary Payment
- Parking Slot
- Staff

By using this system we can track vehicles, parking slot usage, staff work hours and money transactions digitally. This helps to stop people from making mistakes when entering information by hand reduces lost income and makes sure parking spaces are used properly. The database can show how many parking spots are taken in time handle special customers automatically calculate fines and give detailed financial reports. The system can work with MySQL, which's good for organized data and MongoDB which is good, for flexible data. This means the system can be used in ways to suit various needs. The Parking Management System handles all the tasks.

The Parking Management System is very helpful.

2. Introduction and problem statement

1. Urban parking facilities often have problems like:
2. They lose money because they cannot track when people stay long
3. It takes a time to get in and out of the parking facility
4. They do not use the parking spaces in the way for different types of vehicles
5. They do not have information to set the right prices
6. They do not have a system to take care of important customers or reward loyal customers
7. It is hard to manage the schedules and pay of the people who work there
8. They have to issue and track penalties by hand
9. They do not have a system that lets people reserve a spot ahead of time

This project tries to fix these problems by creating a system that keeps track of what's happening in the parking facility calculates fees automatically and helps people make decisions based on the information they have, about Urban parking facilities and how they work. Urban parking facilities will be able to work with this new system.

3. Project objective

The main goal here is to create a place that handles everything about a parking event. This means taking care of things from when a user signs up and enters a parking lot to figuring out how much they owe, processing payments and dealing with penalties.

Specific things we want to do are:

1. Create a simple and organized database plan for all parking related work
2. Use two types of database systems. One that uses tables and one that does not
3. Keep an eye on which parking spots are taken in time across many parking lots
4. Automatically calculate how much someone owes based on how they park and what kind of spot they use
5. Let people reserve spots ahead of time and handle what happens when their reservation runs out
6. Keep track of staff information their work schedules and how much we pay them
7. Track when people get penalties for staying long or breaking rules
8. Make reports about how things are running and how much money we are making
9. Give benefits to important customers and track how often they use our services

10. Make sure our data is safe by letting certain people access certain information, about parking events and the database systems we use for parking operations.

4. System requirement analysis

The system is designed to handle high-concurrency environments. Key requirements were gathered through stakeholder surveys (see Annex: survey_questions.pdf).

4.1 functional requirement

Here is the list of things that the system can do:

- The system lets users register and then authenticate who they are.
- Users can register than one vehicle.
- You can check if a parking slot is available now.
- The system allows users to book a parking slot change their booking or cancel it if they need to.
- Users can book a slot ahead of time. The booking will expire after a certain time.
- The system calculates how much the user has to pay based on how they park their vehicle.
- The system lets users pay with cash, card, Telebirr or mobile money.
- If a user parks their vehicle for long or breaks the rules the system will issue a penalty.
- The system helps manage the schedules of staff members and calculate their salaries.
- Some customers are very important. They get special benefits and discounts.
- The system can generate reports that show things, like how much money was made how often the parking slots were used and how many penalties were issued.

4.2 non functional requirement

- We need to follow the rules for transactions this is called ACID compliance for financial transactions.
- When we check if something is available we need to get a response in under 2 seconds for availability checks.
- Our critical operations need to be working all the time so we aim for 99.9% uptime for critical operations.
- We have to keep records for a long time so we do data retention for 7 years, for these financial records.
- We also need to control who can do what. We use role-based access control with audit logging for role-based access control.

5. Data collection methodology

We did a survey to gather requirements. It was a survey with 10 pages and we sent it to a lot of people including:

1. Customers. They answered sections A and D and we got 50 responses from them
2. Staff and attendants. They answered section B and we got 15 responses from them
3. Management. They answered sections C and E. We got 5 responses from them

The survey had some really important findings, such as:

Most of our customers 68 percent of them like to use mobile money like Telebirr to make payments

Some of our users 45 percent of them are willing to pay for a VIP subscription, which would cost between 500 and 1000 birr per month

Our staff said that the North Zone is the area with the congestion

Our management wants to make 5,000 birr every day

The common problem we have is people overstaying, which accounts for 72 percent of all penalties

The things people told us in the survey helped us decide:

- How to charge per hour for different types of parking slots
- How to handle VIP subscriptions, including the logic and discount rates for VIP
- Which location zone and floor number to assign to each parking spot
- When the staff should start and end their shifts
- How to calculate the penalty amount when someone breaks the rules

6. Entity relationship diagram logic

The architecture follows a star-like schema where the Booking entity acts as the Central hub, connecting User, Vehicle, and Parking Slot. This ensures that every transaction is traceable to a specific individual and physical asset.

6.1 core entities and relations

User (1) ————— (M) Vehicle One user can own multiple vehicles

User (1) ————— (M) Booking One user can make multiple bookings

User (1) ————— (M) Reservation One user can make multiple reservations

User (1) ————— || (1) Staff One user can be a staff member

Parking Lot (1) ————— (M) Parking Slot One lot contains multiple slots

Parking Slot (1) —————(M) Booking One slot can have multiple bookings

Parking Slot (1) —————(M) Reservation One slot can have multiple reservations

Booking (1) —————|| (1) Payment Each booking has one payment

Booking (1) ————— (M) Penalty One booking can have multiple penalties

Staff (1) ————— (M) Salary Payment One staff receives multiple salaries

6.2 Relationship cardinality

- User to Vehicle: One-to-Many (cascade delete)
- User to Booking: One-to-Many (restrict delete)
- Parking Lot to Parking Slot: One-to-Many (cascade delete)
- Booking to Payment: One-to-One (unique constraint)
- Booking to Penalty: One-to-Many (optional)

7. Relation database schema design

The schema utilizes MySQL data types effectively, employing ENUM types for status tracking and DECIMAL for financial accuracy. Foreign key constraints maintain referential integrity.

7.1 complete list of table

Valid role and status values

- DEFAULT values: created_at, status defaults
- ON DELETE CASCADE: Automatic cleanup of dependent records

Table name	Primary key	Foreign keys	Row count
User	user_id	-	6
Vehicle	vehicle_id	user_id	6
Parking Lot	lot_id	-	3
Parking slot	slot_id	lot_id	10
Staff	staff_id	user_id	2
Booking	booking_id	user_id, slot_id, vehicle_id	6
Payment	payment_id	booking_id	4
Reservation	reservation_id	user_id, slot_id, vehicle_id	3
Penalty	penalty_id	booking_id	2
Salary_payment	salary_id	staff_id	2

7.2 Data type selection rationale

- INT/AUTO_INCREMENT: Surrogate keys for efficient joins
- VARCHAR(20) for phone_number: E.164 format support
- ENUM for status fields: Constrained values, storage efficiency
- DECIMAL(10,2) for financial fields: Exact decimal precision
- DATETIME for timestamps: Timezone-aware tracking
- TIME for shift times: Duration-agnostic time storage

8. Database normalization and integrity

To ensure data redundancy is minimized, the database is normalized to the Third Normal Form (3NF) / BCNF. This structure prevents update anomalies and ensures that non-key attributes are functionally dependent only on the primary key.

8.1 Normalization Process

Step 1: Unnormalized Form (UNF)

Booking(user_id, user_name, user_phone, slot_number, slot_type, price, plate_number, start_time, end_time, total_fee)

Step 2: First Normal Form (1NF) - Remove repeating groups

Separate repeating groups into individual rows/ tables

Step 3: Second Normal Form (2NF) - Remove partial dependencies

Move slot_type, price to ParkingSlot table

Move user_name, user_phone to User table

Step 4: Third Normal Form (3NF) - Remove transitive dependencies No transitive dependencies remain

Step 5: Boyce-Codd Normal Form (BCNF)

Every determinant is a candidate key - satisfied

8.2 Integrity Constraints Implemented

- Primary Keys: Uniquely identify each row
- Foreign Keys: Maintain referential integrity
- UNIQUE constraints: email, phone_number, plate_number, slot_number
- NOT NULL constraints: Required fields
- CHECK constraints:

9. **DATA DICTIONARY: USER & VEHICLE**

9.1 **User Table**

Field	Type	Description
user_id	INT(PK, AUTO_INCREMENT)	Unique user identifier
password_hash	VARCHAR(255) NOT NULL	Hashed password for security
role	ENUM	Admin/manager/customer/staff
full_name	VARCHAR(100) NOT NULL	User's full legal name
email	VARCHAR(100) UNIQUE	Contact email(login credential)
phone_number	VARCHAR(20) UNIQUE	Contact phone (SMS notifications)
is_vip	BOOLEAN DEFAULT FALSE	VIP status flag
created_at	DATETIME DEFAULT NOW()	Account creation timestamp
status	ENUM DEFAULT 'active'	active/inactive/banned

9.2 **Vehicle Table**

Field	Type	Description
vehicle_id	INT (PK, AUTO_INCREMENT)	Unique Vehicle
plate_number	VARCHAR(28) UNIQUE	License plate(unique per vehicle)
user_id	INT(FK) NOT NULL	Owner reference (links to user)
vehicle_type	ENUM	car/motorcycle/truck/bus/bicycle
brand	VARCHAR(50)	Manufacturer (Toyota, Honda, etc.)
color	VARCHAR(30)	Vehicle color

registered_at	DATETIME DEFAULT NOW()	Registration timestamp
---------------	------------------------	------------------------

10. **DATA DICTIONARY: PARKINGLOT & PARKINGSLOT**

10.1 **Parking Lot Table**

Field	Type	Description
lot_id	INT (PK, AUTO_INCREMENT)	Unique lot identifier
lot_name	VARCHAR(100) NOT NULL	Display name (e.g., "Main Campus")
location	VARCHAR(255) NOT NULL	Physical address
total_slots	INT NOT NULL	Total capacity
available_slots	INT NOT NULL	Currently available
opening_time	TIME NOT NULL	Daily opening hour
closing_time	TIME NOT NULL	Daily closing hour
status	ENUM DEFAULT 'open'	open/closed/maintenance/full

10.2 ParkingSlot Table

Field	Type	Description
slot_id	INT (PK, AUTO_INCREMENT)	Unique slot identifier
slot_number	VARCHAR(10) UNIQUE	Physical label (e.g., "A01")
lot_id	INT (FK) NOT NULL	Parent parking lot
type	ENUM	standard/vip/handicap/compact/large/electric
price_per_hour	DECIMAL(10,2) NOT NULL	Hourly rate in birr
is_occupied	BOOLEAN DEFAULT FALSE	Current occupancy flag
location_zone	VARCHAR(50)	North/South/East/West
floor_number	INT	Multi-level support
status	ENUM DEFAULT 'available'	available/occupied/maintenance/reserved

11. **DATA DICTIONARY: BOOKING & RESERVATION**

11.1 **Booking Table**

Field	Type	Description
booking_id	INT (PK, AUTO INCREMENT)	Unique booking identifier
user_id	INT (FK) NOT NULL	Customer reference
slot_id	INT (FK) NOT NULL	Parking space reference
vehicle_id	INT (FK) NOT NULL	Vehicle used
start_time	DATETIME NOT NULL	Parking start timestamp
end_time	DATETIME	Parking end timestamp(NULL if active)
duration	DECIMAL(10,2)	Calculated hours (end - start)
total_fee	DECIMAL(10,2)	Calculated amount (duration * price)
payment_status	ENUM DEFAULT 'pending'	pending/paid/failed/refunded
booking_status	ENUM DEFAULT 'active'	active/completed/cancelled/expired/no_show
created_at	DATETIME DEFAULT NOW()	Booking creation time

11.2 **Reservation Table**

Field	Type	Description
reservation_id	INT (PK, AUTO INCREMENT)	Unique reservation identifier
user_id	INT (FK) NOT NULL	Customer reference
slot_id	INT (FK) NOT NULL	Reserved slot
vehicle_id	INT (FK) NOT NULL	Vehicle to be used
reservation_time	DATETIME DEFAULT NOW()	When reservation was made
expiry_time	DATETIME NOT NULL	When reservation expires
status	ENUM DEFAULT 'active'	active/expired/cancelled/completed

12. **DATA DICTIONARY: PAYMENT & PENALTY**

12.1 **Payment Table**

Field	Type	Description
payment_id	INT (PK, AUTO_INCREMENT)	Unique payment identifier
booking_id	INT (FK) UNIQUE NOT NULL	Associated booking (one-to-one)
amount	DECIMAL(10,2) NOT NULL	Payment amount in birr
payment_method	ENUM NOT NULL	cash/credit_card/debit_card/mobile_money/telebirr
payment_date	DATETIME DEFAULT NOW()	Transaction timestamp
payment_status	ENUM DEFAULT 'pending'	pending/completed/failed/refunded
transaction_id	VARCHAR(100) UNIQUE	External reference (gateway ID)

12.2 **Penalty Table**

Field	Type	Description
penalty_id	INT (PK, AUTO_INCREMENT)	Unique penalty identifier
booking_id	INT (FK) NOT NULL	Related booking
amount	DECIMAL(10,2) NOT NULL	Penalty amount in birr
reason	ENUM NOT NULL	overstay/wrong_parking/damage/no_payment/other
issued_at	DATETIME DEFAULT NOW()	When penalty was issued
status	ENUM DEFAULT 'pending'	pending/paid/waived

13. **DATA DICTIONARY: STAFF & SALARYPAYMENT**

13.1 **Staff Table**

Field	Type	Description
staff_id	INT (PK, AUTO INCREMENT)	Unique staff identifier
user_id	INT (FK) NOT NULL	Linked User account
role	ENUM NOT NULL	attendant/supervisor/manager/security/cleaner
shift_start	TIME NOT NULL	Daily shift start time
shift_end	TIME NOT NULL	Daily shift end time
status	ENUM DEFAULT 'active'	active/on_leave/inactive

13.2 **SalaryPayment Table**

Field	Type	Description
salary_id	INT (PK, AUTO INCREMENT)	Unique salary identifier
staff_id	INT (FK) NOT NULL	Staff member reference
amount	DECIMAL(10,2) NOT NULL	Salary amount in birr
payment_date	DATE NOT NULL	When salary was paid
payment_method	ENUM NOT NULL	cash/bank_transfer/telebirr
payment_status	ENUM DEFAULT 'pending'	pending/completed/failed
month_year	DATE NOT NULL	Period covered (first of month)

14. **IMPLEMENTATION: DDL AND DML**

The database was instantiated using standard SQL scripts (see Appendix A for complete schema). Sample datasets were injected to simulate real-world scenarios including active parkers, completed stays, and slots currently under maintenance.

14.1 **DDL Summary (schema.sql)**

- 10 CREATE TABLE statements with all constraints
- Foreign key relationships with ON DELETE CASCADE/SET NULL
- ENUM definitions for status and type fields
- UNIQUE constraints for email, phone, plate_number, slot_number

- Indexes on frequently queried columns (status, is_occupied)

14.2 DML Summary (data insertion)

- **User:** 6 records (customers, admin, manager, staff)
- **Vehicle:** 6 records (cars, motorcycle, truck)
- **ParkingLot:** 3 records (multiple locations)
- **ParkingSlot:** 10 records (various types and zones)
- **Staff:** 2 records (attendant, supervisor)
- **Booking:** 6 records (active and completed)
- **Payment:** 4 records (various methods)
- **Reservation:** 3 records (active reservations)
- **Penalty:** 2 records (overstay, no_payment)
- **SalaryPayment:** 2 records (monthly salaries)

14.3 Technology Stack

- **MySQL 8.0:** Relational database engine
- **MongoDB 6.0:** NoSQL document store
- **Draw.io:** ER Diagram design
- **GitHub:** Version control and collaboration

15. ANALYSIS: INFORMATION RETRIEVAL

Standard retrieval queries allow staff to quickly identify available slots by price or location, while customers can view their active parking status and historical records.

15.1 Key Retrieval Queries (See queries.sql for complete list)

- **Find all available slots:** SELECT with status = 'available'
- **Find VIP slots only:** Filter by type = 'vip'
- **User booking history:** JOIN User + Booking + Vehicle
- **Currently occupied slots:** is_occupied = TRUE with customer info
- **Vehicle lookup by plate:** Exact match on plate_number

15.2 Sample Query: Available VIP Slots

```
SELECT ps.slot_number, ps.price_per_hour, pl.lot_name, pl.location
FROM ParkingSlot ps
JOIN ParkingLot pl ON ps.lot_id = pl.lot_id
WHERE ps.type = 'vip' AND ps.status = 'available';
```

15.3 Sample Query: Customer Active Bookings

```
SELECT b.booking_id, b.start_time, ps.slot_number, pl.lot_name
FROM Booking b
JOIN ParkingSlot ps ON b.slot_id = ps.slot_id
JOIN ParkingLot pl ON ps.lot_id = pl.lot_id
WHERE b.user_id = [user_id] AND b.booking_status = 'active';
```

16. ANALYSIS: FINANCIAL REPORTING

The system generates automated revenue reports, aggregating income by date and payment channel. This allows management to monitor the facility's economic health and identify peak revenue periods.

16.1 Revenue Reports Implemented

- Daily revenue summary (by date)
- Payment method breakdown (cash vs card vs Telebirr)
- Revenue by slot type (VIP vs Standard)
- Monthly revenue trends
- Revenue forecast from active bookings

16.2 Sample Query: Daily Revenue Report

```
SELECT DATE(payment_date) AS date,
       SUM(amount) AS daily_revenue,
       COUNT(payment_id) AS num_transactions,
       ROUND(AVG(amount), 2) AS avg_transaction
FROM Payment
WHERE payment_status = 'completed'
GROUP BY DATE(payment_date)
```

ORDER BY date DESC;

16.3 Sample Query: Revenue by Slot Type

```
SELECT ps.type,  
       COUNT(p.payment_id) AS num_payments,  
       SUM(p.amount) AS total_revenue,  
       ROUND(AVG(p.amount), 2) AS avg_payment  
FROM Payment p  
      JOIN Booking b ON p.booking_id = b.booking_id    JOIN  
ParkingSlot ps ON b.slot_id = ps.slot_id  
      WHERE p.payment_status = 'completed'  
      GROUP BY ps.type  
      ORDER BY total_revenue DESC;
```

17. ANALYSIS: OPERATIONAL METRICS

By calculating the ratio of occupied to total slots per floor or per lot, the system provides a utilization index. This helps in directing traffic to less crowded zones, improving the overall flow of the facility.

17.1 Operational Metrics Implemented

- Utilization rate by floor (occupied/total * 100)
- Utilization rate by parking lot
- Average parking duration
- Peak hour analysis (by hour of day)
- Staff performance (bookings handled per staff)
- Slot turnover rate

17.2 Sample Query: Utilization by Parking Lot

```
SELECT pl.lot_name,  
       pl.total_slots,  
       pl.available_slots,  
       (pl.total_slots - pl.available_slots) AS occupied_slots,
```



```

        ROUND((pl.total_slots - pl.available_slots) * 100.0 / pl.total_slots, 2) AS
utilization_percent
FROM ParkingLot pl;

```

17.3 Sample Query: Peak Hour Analysis

```

SELECT HOUR(start_time) AS hour_of_day,
        COUNT(booking_id) AS bookings_started,
SUM(total_fee) AS revenue
FROM Booking
WHERE payment_status = 'paid'
GROUP BY HOUR(start_time)
ORDER BY revenue DESC
LIMIT 5;

```

18. ANALYSIS: BEHAVIORAL PATTERNS

The database tracks user frequency, identifying "Frequent Parkers." This intelligence allows the business to target specific users for loyalty programs or VIP upgrades.

18.1 Behavioral Metrics Implemented

- Customer lifetime value (total spent per customer)
- Visit frequency (bookings per time period)
- Preferred parking hours (time-of-day analysis)
- Preferred slot types
- Cancellation rate analysis
- No-show rate for reservations

18.2 Sample Query: Top Customers by Lifetime Value

```

SELECT u.user_id,
        u.full_name,
        u.is_vip,
        COUNT(b.booking_id) AS total_visits,
        SUM(b.total_fee) AS lifetime_value,

```

```

        ROUND(AVG(b.total_fee), 2) AS avg_per_visit
FROM User u

JOIN Booking b ON u.user_id = b.user_id

WHERE b.payment_status = 'paid'

GROUP BY u.user_id

ORDER BY lifetime_value DESC

LIMIT 10;

```

18.3 Sample Query: Reservation Conversion Rate

```

SELECT COUNT(r.reservation_id) AS total_reservations,
       COUNT(CASE WHEN r.status = 'completed' THEN 1 END) AS converted,
       COUNT(CASE WHEN r.status = 'expired' THEN 1 END) AS expired,
       ROUND(COUNT(CASE WHEN r.status = 'completed' THEN 1 END) * 100.0 /
COUNT(r.reservation_id), 2) AS conversion_rate
FROM Reservation r;

```

19. VIP PROGRAM MODELING

The system includes logic for a VIP tier, offering premium slots and discounted rates. Data analysis compares standard revenue against potential loyalty earnings to justify the program's expansion.

19.1 VIP Features Implemented

- is_vip boolean flag in User table
- VIP-only parking slots (type = 'vip')
- Discount logic for total_fee calculation
- Priority reservation windows
- VIP customer analytics

19.2 Sample Query: VIP vs Non-VIP Comparison

```

SELECT u.is_vip,
       COUNT(b.booking_id) AS total_bookings,
       SUM(b.total_fee) AS total_revenue,

```

```

        ROUND(AVG(b.total_fee), 2) AS avg_booking_value
FROM User u
    JOIN Booking b ON u.user_id = b.user_id
    WHERE b.payment_status = 'paid'
    GROUP BY u.is_vip;

```

19.3 VIP Recommendation Logic

Based on frequency analysis, users with:

- 5+ bookings per month
- Total monthly spend > 500 birr
- 0 no-shows in last 3 months

→ Eligible for VIP upgrade invitation

20. SECURITY & ACCESS CONTROL

Access is strictly partitioned using role-based access control (RBAC). Users can only interact with their own data, while Managers and Admins have graduated levels of oversight for maintenance and financial auditing.

20.1 Role Definitions

Role	Permissions
Admin	Full system access, user management, financial
Manager	View all reports, manage staff, issue/waive
Staff	Process check-in/out, update slot status, view assigned
Customer	View own bookings, make reservations, process

20.2 Security Features Implemented

- Password hashing (not stored in plain text)
- Audit logging for critical actions
- Session management
- Input validation to prevent SQL injection
- HTTPS for data transmission (future implementation)

20.3 Sample Audit Query

Conceptual audit log (to be implemented)

```
SELECT user_id, action, timestamp, ip_address
FROM AuditLog
WHERE action IN ('DELETE_BOOKING', 'UPDATE_PRICE', 'ISSUE_PENALTY')
ORDER BY timestamp DESC;
```

21. **SCALABILITY & FUTURE WORK**

The design allows for seamless integration with external hardware, such as automated barriers or license plate recognition cameras. Future iterations will focus on mobile-first reservations and dynamic pricing.

21.1 **Scalability Features**

- Indexed columns for fast lookups (status, user_id, booking_status)
- Partitioning by date for large tables
- Connection pooling for high concurrency
- Read replicas for reporting queries

21.2 **Future Enhancements**

Phase 1 (Immediate):

- Mobile application for end users
- SMS notifications for expiry alerts
- Real-time occupancy dashboard

Phase 2 (3-6 months):

- License plate recognition (LPR) integration
- Automated barrier control
- Dynamic pricing based on demand

Phase 3 (6-12 months):

- Predictive occupancy modeling
- Integration with city-wide parking system
- Electric vehicle charging station management

22. CONCLUSION & RECOMMENDATIONS

22.1 Summary of Achievements

- Designed and implemented a 10-table relational database (BCNF normalized)
- Created dual implementation (MySQL + MongoDB)
- Developed 20+ SQL queries for operational and financial analysis
- Implemented VIP loyalty program logic
- Built staff management and payroll tracking
- Designed penalty and reservation systems
- Created comprehensive documentation and user forms

22.2 Key Metrics

Database Size: 10 tables, 6-10 records each (sample data)
Query Response Time: < 100ms (with indexes)

Supported Concurrent Users: 100+ (estimated)

Data Retention Policy: 7 years for financial records

22.3 Recommendations

1. Prioritize mobile payment integration (68% of surveyed customers prefer Telebirr/mobile money) to maximize user convenience.
2. Implement real-time occupancy dashboards at lot entrances to reduce search time and improve facility flow.
3. Launch VIP pilot program with 10% discount for early adopters to test revenue impact before full rollout.
4. Automate penalty calculation and notification to reduce manual errors and increase overstay deterrence.
5. Deploy license plate recognition (LPR) at entry/exit to eliminate manual data entry and speed up processing.
6. Conduct quarterly survey reviews to continuously align pricing and features with customer expectations.

22.4 Final Statement

The Integrated Parking Management System provides a scalable, data-driven framework for modernizing parking operations. The database design successfully addresses all identified requirements and is ready for production deployment with appropriate hardware integration.

23. **REFERENCES**

[1] Elmasri, R. and Navathe, S. (2016). Fundamentals of Database Systems. 7th ed. Pearson.

[2] Oracle Corporation. (2024). MySQL 8.0 Reference Manual.
Available at: <https://dev.mysql.com/doc/refman/8.0/en/>

[3] MongoDB Inc. (2024). MongoDB Manual.
Available at: <https://docs.mongodb.com/>

[4] Chen, P.P. (1976). The Entity-Relationship Model - Toward a Unified View of Data. ACM Transactions on Database Systems, 1(1), pp.9-36.

Codd, E.F. (1970). A Relational Model of Data for Large Shared Data Banks. Communications of the ACM, 13(6), pp.377-387.

APPENDIX: COMPLETE SQL SCRIPTS

The following files are attached separately:

- /mysql/schema.sql - Complete DDL (10 tables with constraints)
- /mysql/queries.sql - 20+ analytical queries
- /mongodb/collections.json - MongoDB document collections
- /mongodb/queries.js - 30+ NoSQL queries
- /diagrams/erd.png - Entity Relationship Diagram
- /diagrams/normalization.pdf - Normalization proof
- /annex/survey_questions.pdf - Data collection instrument
- /annex/sample_forms.pdf - Operational forms templates